

Veri Organizasyonu (289)

Doç. Dr. Ahmet Arif AYDIN

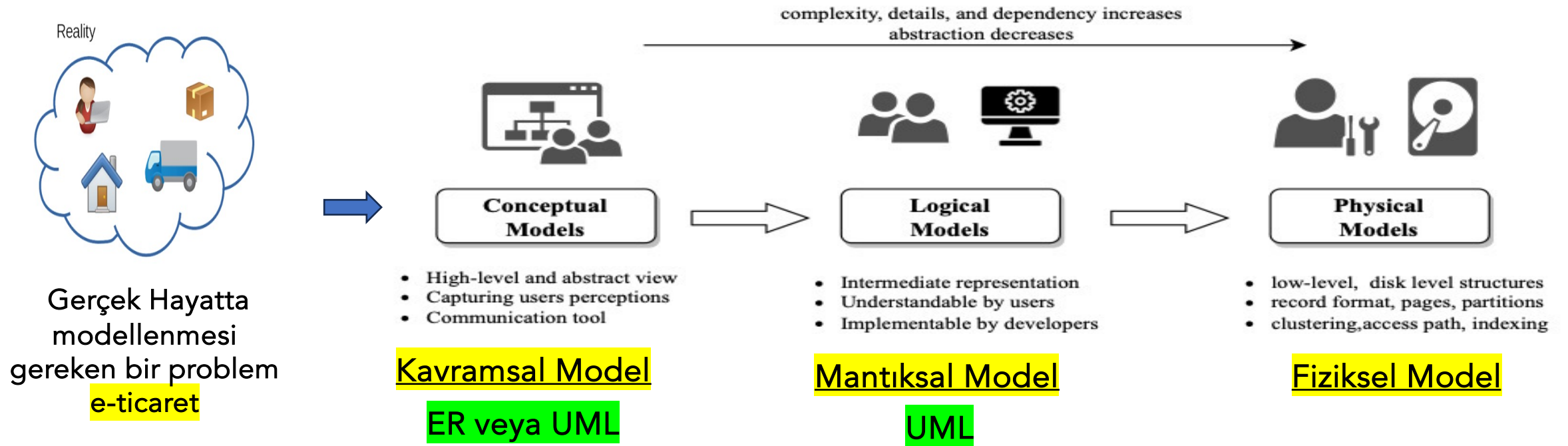
L10– Fiziksel Veri Modeli ve B+ Ağaçları

Güz Dönemi -2024

- UML(Unified Modeling Language) nedir ?
- UML'in kullanım **amaçları** nelerdir?
- UML'in kullanım **alanları** nelerdir?
- UML diagram çeşitlerinin iki ana kategorisini açıklayınız ?
- UML Diagram Çeşitleri
 - **Category 1** ➔ Class, Component, Deployment
 - **Category 2** ➔ Use Case, Activity, Sequence, State Machine

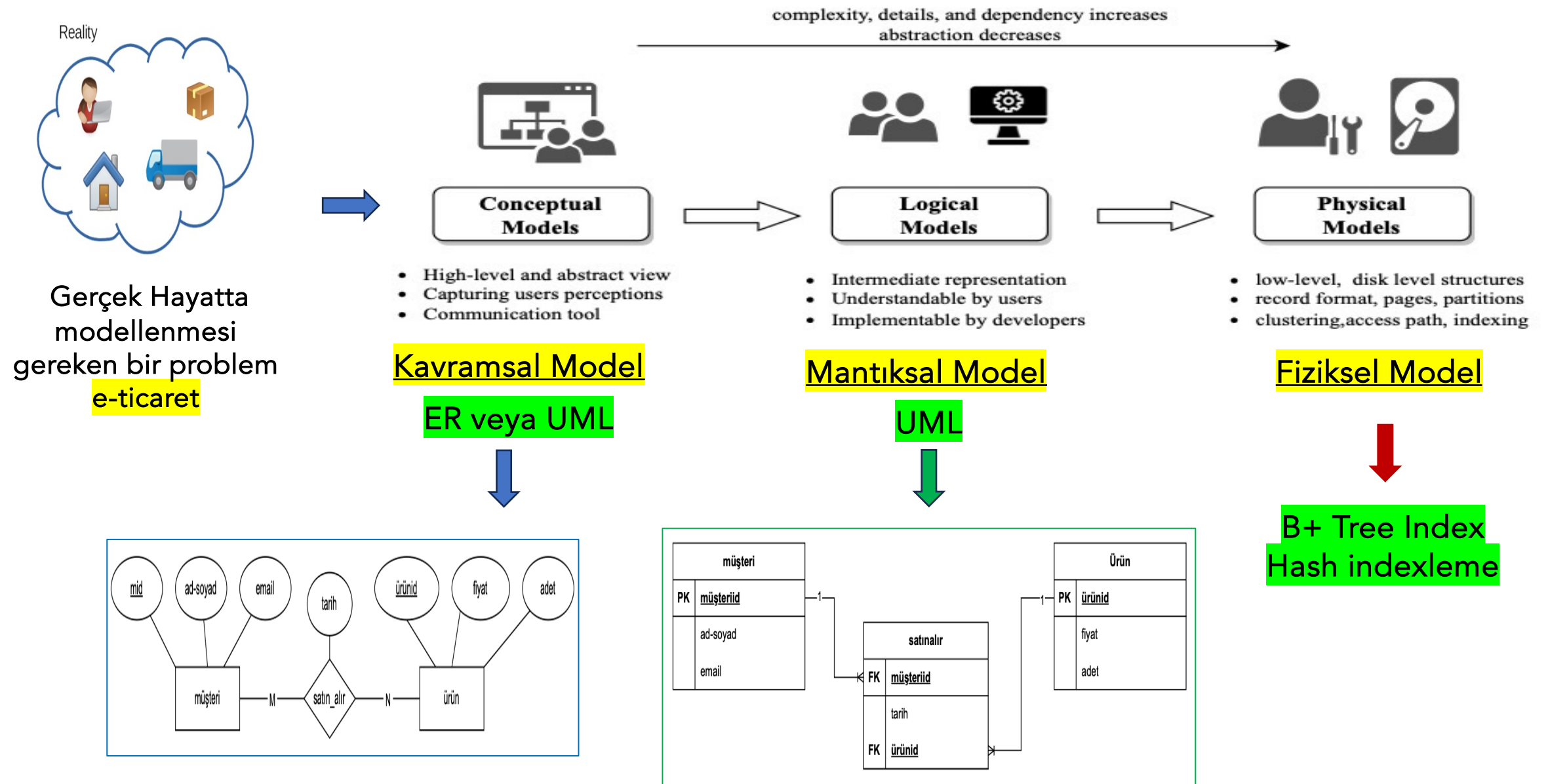


➤ Verinin Modellenmesi

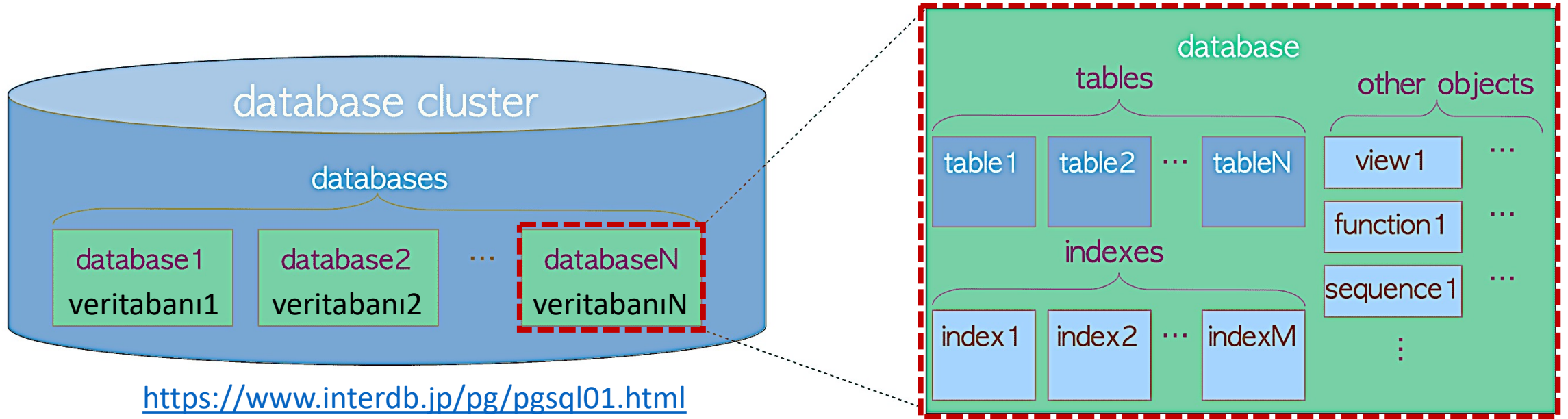


?

➤ Verinin Modellenmesi



➤ Veritabanı Organizasyonu



- Veritabanı yönetim sistemleri işletim sistemlerinin **disk yöneticisi** ile koordineli çalışarak veritabanında kaydedilecek veriyi disk üzerinde depolar.
- Veritabanında bulunana bütün nesneler **object identifiers (OIDs- 4byte integer)** ile belirlenir.
- Bir veritabanında tablolar , index, view, function, sequences, and stored procedure'lar bulunur.

➤ Tablolar ve Verinin Depolanması

Employees Table			
EmployeeId	Name	Email	Department
1	Mark	mark@pragimtech.com	IT
2	John	john@pragimtech.com	HR
3	Sara	sara@pragimtech.com	HR
4	Mary	mary@pragimtech.com	IT
5	Dave	dave@pragimtech.com	IT
...			
...			
1200	Steve	steve@pragimtech.com	HR

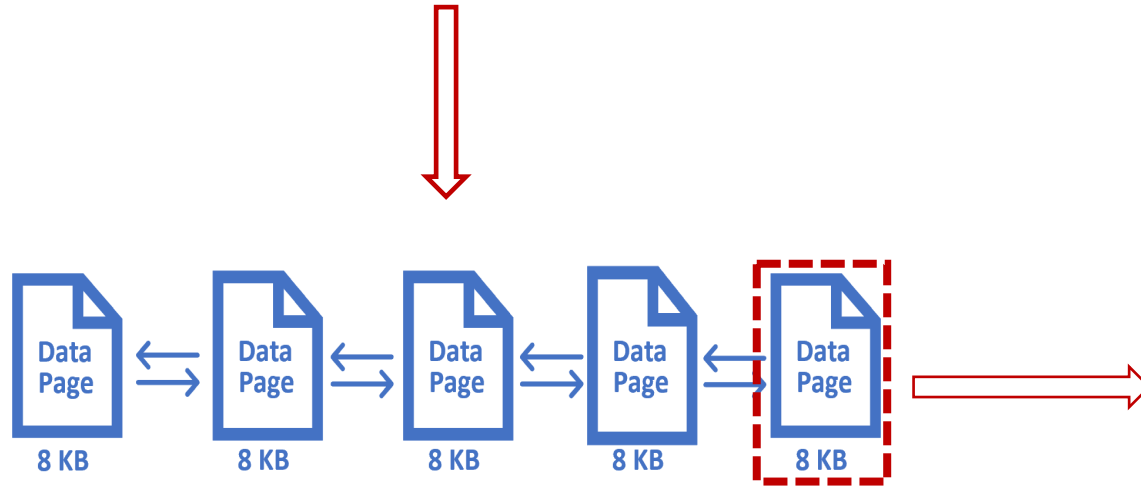
- Veritabanı **parametresi** **sayfa (page)** dır.
- Verinin diske yazılan ve sonrasında ihtiyaç halinde okunduğu her bir birim **sayfa (page)** olarak adlandırılır.

➤ Tablo ve Verinin Depolanması: Sayfa

Employees Table			
EmployeeId	Name	Email	Department
1	Mark	mark@pragimtech.com	IT
2	John	john@pragimtech.com	HR
3	Sara	sara@pragimtech.com	HR
4	Mary	mary@pragimtech.com	IT
5	Dave	dave@pragimtech.com	IT
...			
...			
1200	Steve	steve@pragimtech.com	HR

➤ Veritabanı parametresi **sayfa (page)** dir.

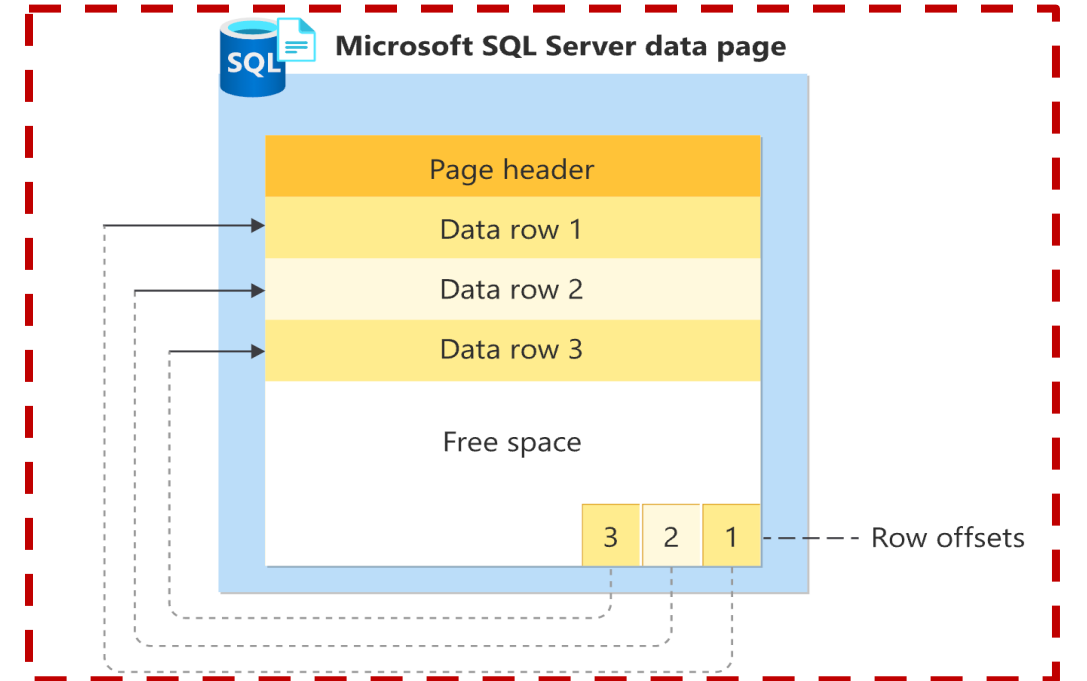
➤ Verinin diske yazılan ve sonrasında ihtiyaç halinde okunduğu her bir birim **sayfa (page)** olarak adlandırılır.



Sayfa boyutu veritabanına göre değişiklik gösterir

PostgreSQL (8KB), MySQL (16KB), Oracle (4KB)

<https://www.pragimtech.com/blog/sql-optimization/how-is-data-stored-in-sql-database/>

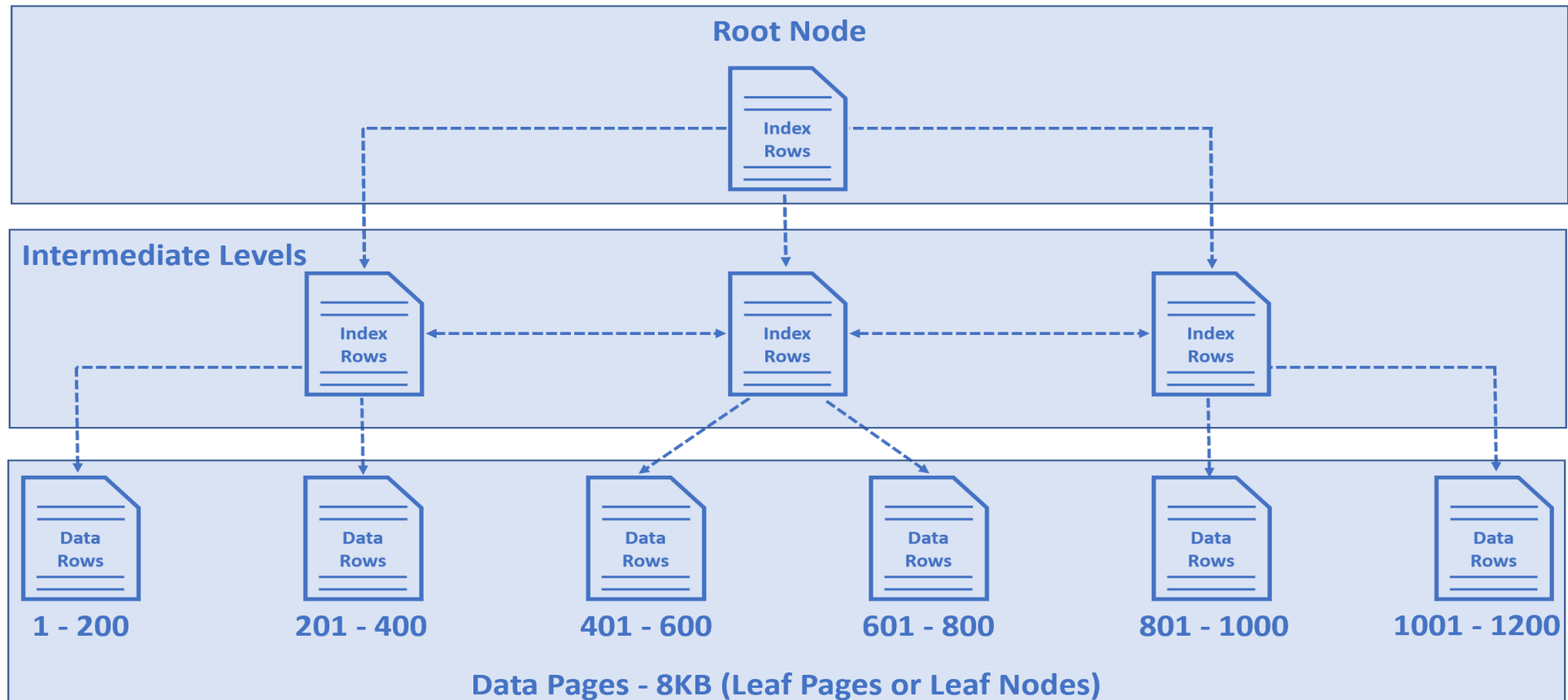


➤ Tablolar ve Verinin Depolanması: B+ Ağacı

Employees Table			
EmployeeId	Name	Email	Department
1	Mark	mark@pragimtech.com	IT
2	John	john@pragimtech.com	HR
3	Sara	sara@pragimtech.com	HR
4	Mary	mary@pragimtech.com	IT
5	Dave	dave@pragimtech.com	IT
...			
...			
1200	Steve	steve@pragimtech.com	HR

<https://www.pragimtech.com/blog/sql-optimization/how-is-data-stored-in-sql-database/>

İlişkisel Veritabanı Yönetim sistemlerinde (RDBMS)
varsayılan (default) veri depolama mekanizması
B+ Ağacıdır

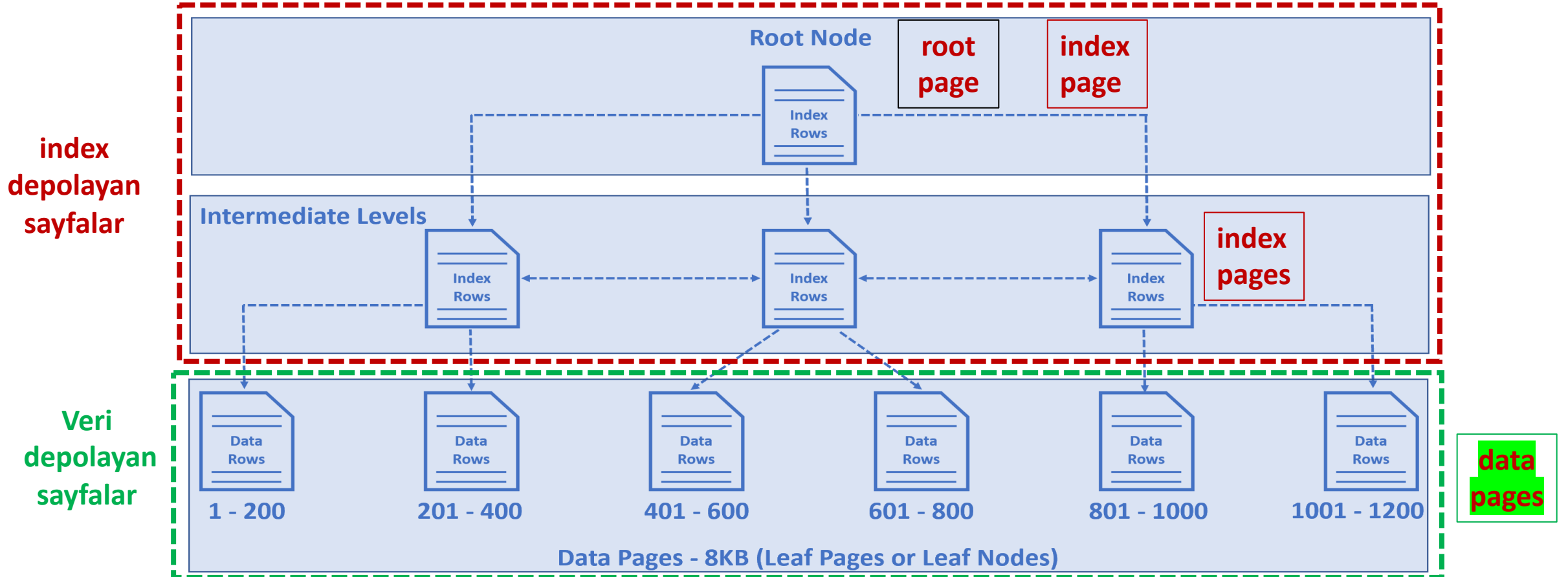


➤ Tablolar ve Verinin Depolanması: B+ Ağacı

Employees Table			
EmployeeId	Name	Email	Department
1	Mark	mark@pragimtech.com	IT
2	John	john@pragimtech.com	HR
3	Sara	sara@pragimtech.com	HR
4	Mary	mary@pragimtech.com	IT
5	Dave	dave@pragimtech.com	IT
...			
...			
1200	Steve	steve@pragimtech.com	HR

<https://www.pragimtech.com/blog/sql-optimization/how-is-data-stored-in-sql-database/>

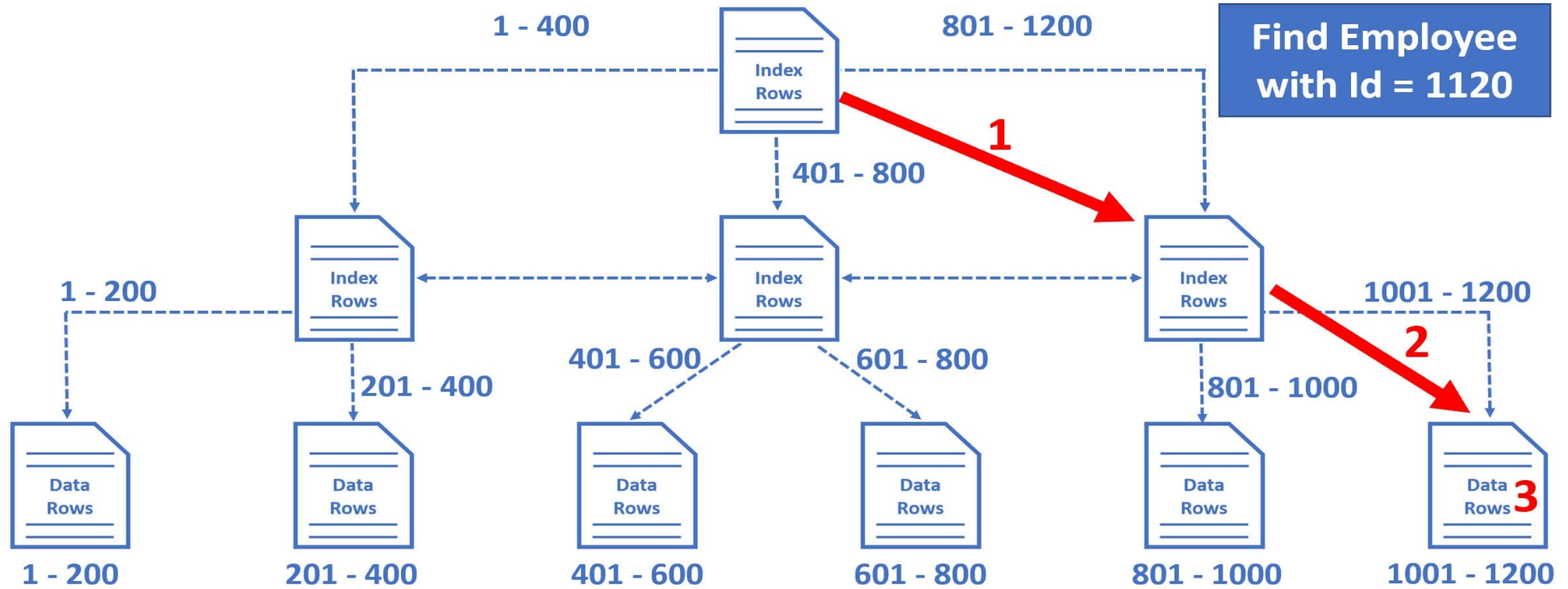
İlişkisel Veritabanı Yönetim sistemlerinde (RDBMS)
varsayılan (default) veri depolama mekanizması
B+ Ağacıdır



➤ Tablolar ve Verinin Depolanması: B+ Ağacı

Employees Table			
EmployeeId	Name	Email	Department
1	Mark	mark@pragimtech.com	IT
2	John	john@pragimtech.com	HR
3	Sara	sara@pragimtech.com	HR
4	Mary	mary@pragimtech.com	IT
5	Dave	dave@pragimtech.com	IT
...			
...			
1200	Steve	steve@pragimtech.com	HR

<https://www.pragimtech.com/blog/sql-optimization/how-is-data-stored-in-sql-database/>



Veritabanının parametresi **sayfa (page)** dir. Sorgulama yapıldığında **veri sayfa ile gönderilir**

➤ Disk Alanı Yöneticisi ve Görevleri



Disk Alanı Yönetimi, DBMS'nin **en alt katmanıdır** ve **disk üzerindeki alanın yönetiminden** sorumludur.

- Sayfaları (page) diskteki konumlarla eşleştirir
- **Page I/O gerçekleştirilir:**
 - Sayfaları **diskten → belleğe** yükler
 - Sayfaları **bellekten → diske** kaydeder
- Sayfalarda güncelleme ve yazmaları yönetir.

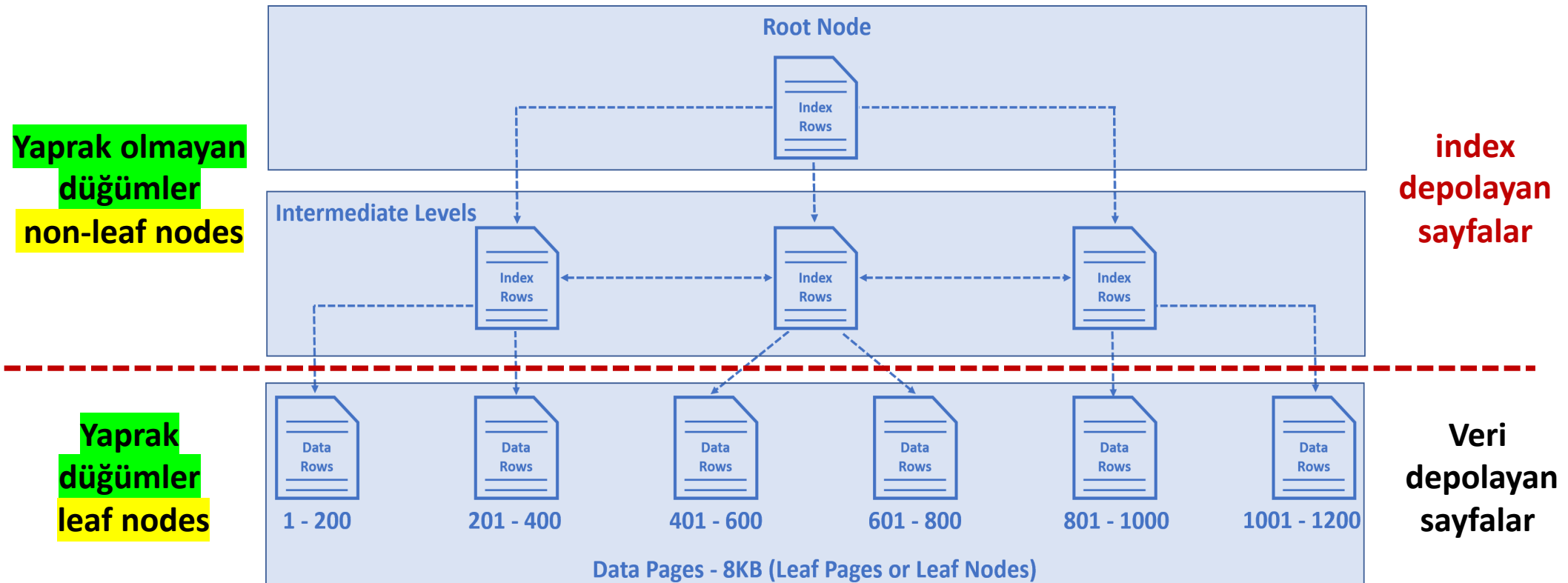
<https://cs186berkeley.net/notes/note3/>

➤ B+ Ağacının özellikleri

Employees Table			
EmployeeId	Name	Email	Department
1	Mark	mark@pragimtech.com	IT
2	John	john@pragimtech.com	HR
3	Sara	sara@pragimtech.com	HR
4	Mary	mary@pragimtech.com	IT
5	Dave	dave@pragimtech.com	IT
...			
1200	Steve	steve@pragimtech.com	HR

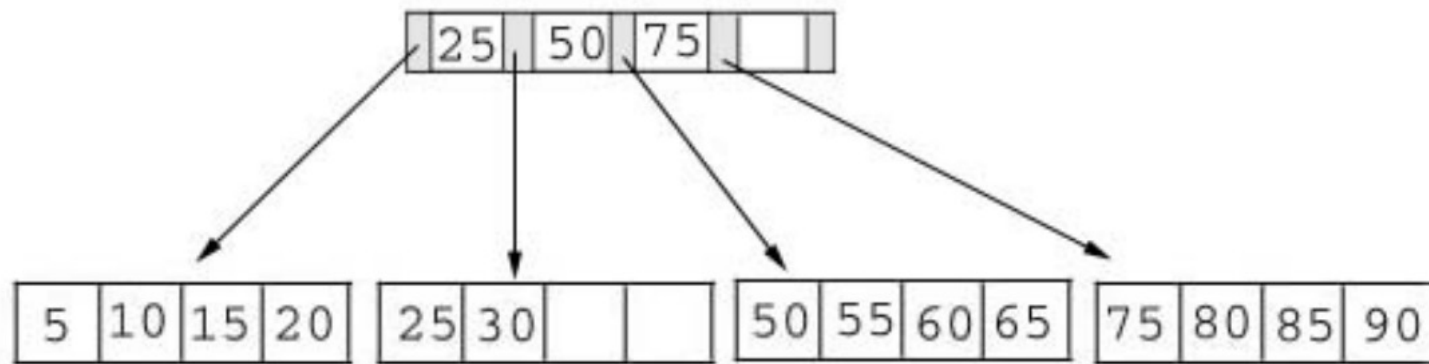
İlişkisel Veritabanı Yönetim sistemlerinde (RDBMS)
varsayılan (default) veri depolama mekanizması B+ Ağacıdır

- B+ ağacının esnek ve dinamik bir yapısı bulunmaktadır
- B+ ağacındaki her bir düğüm disk üzerinde bir sayfadır.



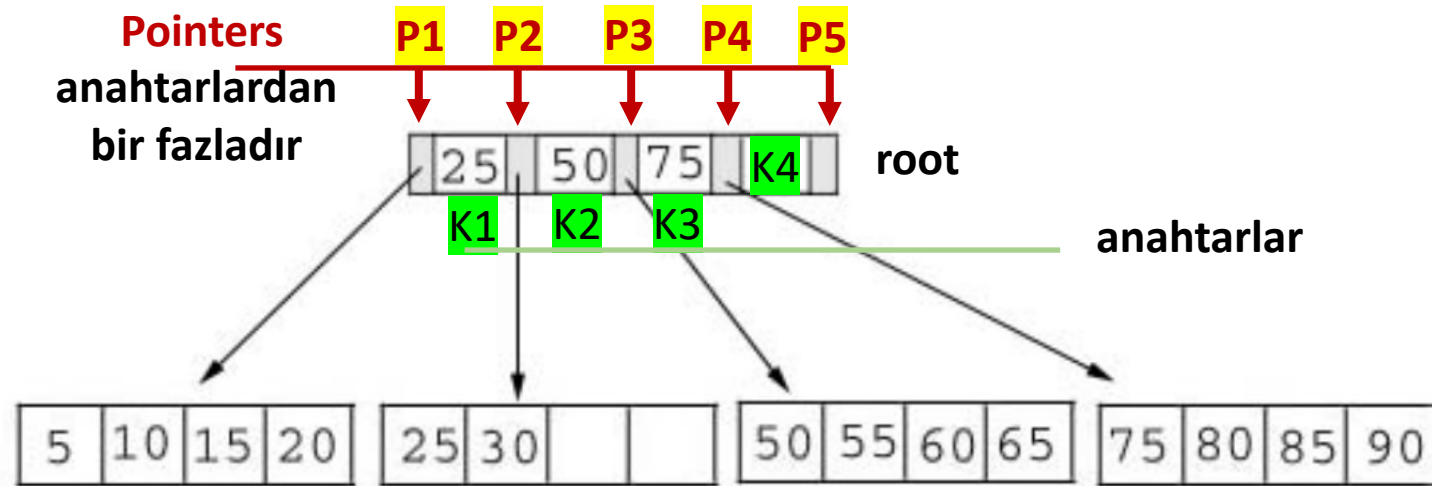
➤ B+ Ağacının Özellikleri

- B+ ağacında **bir adet kök düğüm (root node)** bulunur.
- **Kök düğüm hariç her bir düğüm için %50 doluluk şartı** vardır.
- Düğümlerde saklanacak veri aşağıdaki ifadeye göre belirlenir:
 - **$d \leq \text{düğümde eleman sayısı} \leq 2d$**
 - **d parametresi (order of tree)** bir düğümde bulunabilecek minimum kayıt sayısını belirtir.
 - d bir sayfaya kaydedilen kayıtların kapladığı alana göre belirlenir



➤ B+ Ağacının Özellikleri

- B+ ağacının yapısında bulunan işaretçiler (**pointer**) yardımıyla aramalar yönlendirilir.
- **Leaf nodes** (yaprak düğümleri) kaydedilen verileri içermektedir.
- B+ ağacının yapısı eklenen veya silinen anahtara göre dinamik olarak değişir.



➤ B+ Ağacının Özellikleri

➤ B+ ağacının yüksekliği ve kapasitesi

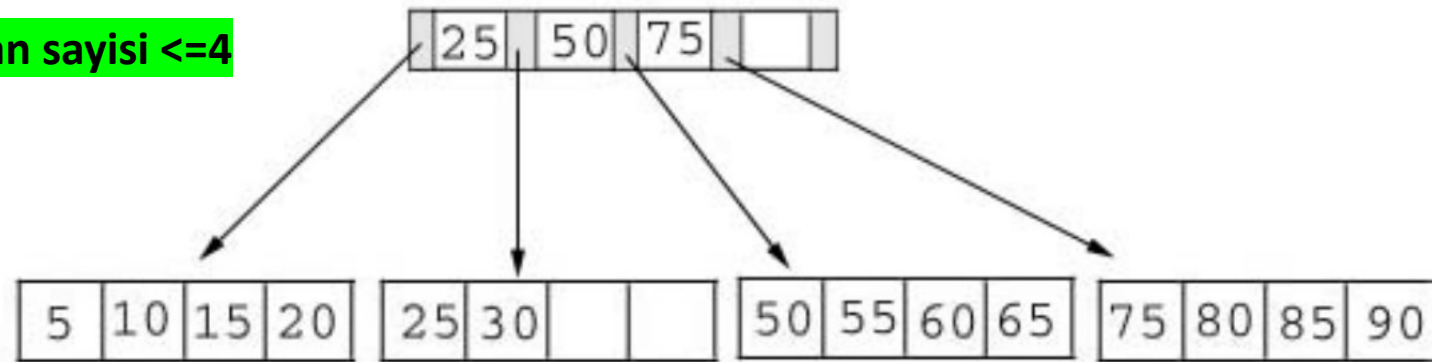
➤ $h = \log_k N$ formulu ile hesaplanır

➤ $k = \text{düğümün kapasitesi}$, $N = \text{kayıt sayısı}$

➤ $N = k^h$

d=2

$2 \leq \text{düğümdeki elaman sayısı} \leq 4$



h= 2

K=4

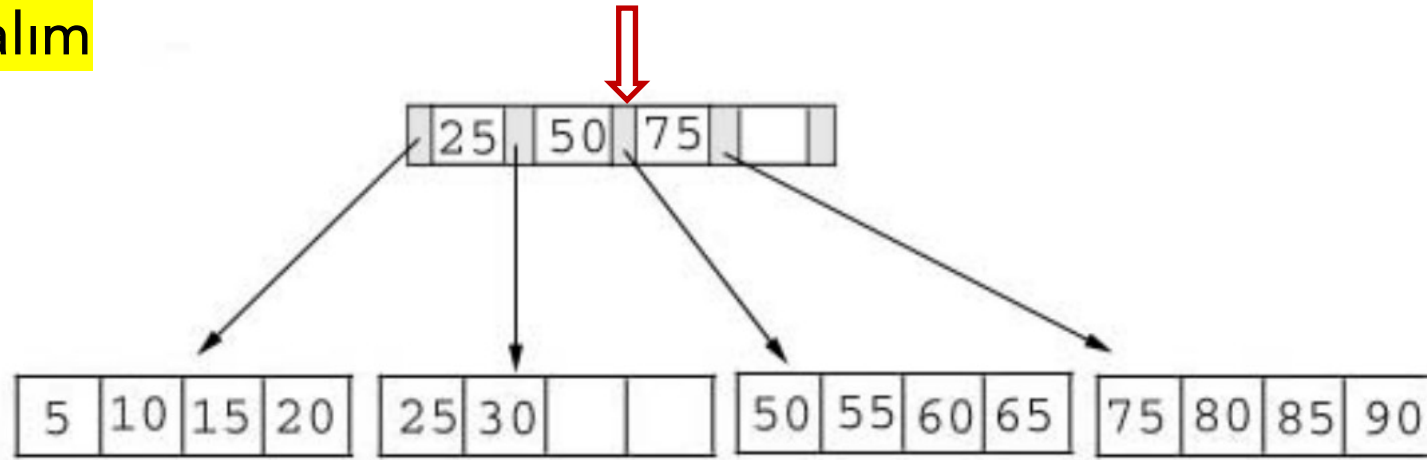
N= 4²

➤ B+ Ağacı: Arama (Search)

1. B+ ağacında arama işlemine **root (kök düğüm)** ile başlanır
2. Düğümün türüne göre a veya b seçeneği çalıştırılır
 - a) Arama yapılan düğüm **yaprak olmayan düğüm (non-leaf node)** ise
 - Anahtar değerine göre düğümde arama yapılır
 - **Aranılan anahtar değerden küçük olan en yüksek değerli anahtardan sonraki işaretleyicinin gösterdiği düğüme gidilir.**
 - Arama yapılan düğüm güncellenir
 - 2.adıma geri dönülür.
 - b) Arama yapılan düğüm bir **yaprak düğüm (leaf node)** ise
 - **aranılan değer bu düğümdedir veya yoktur.**

➤ B+ Ağacı: Arama (Search)

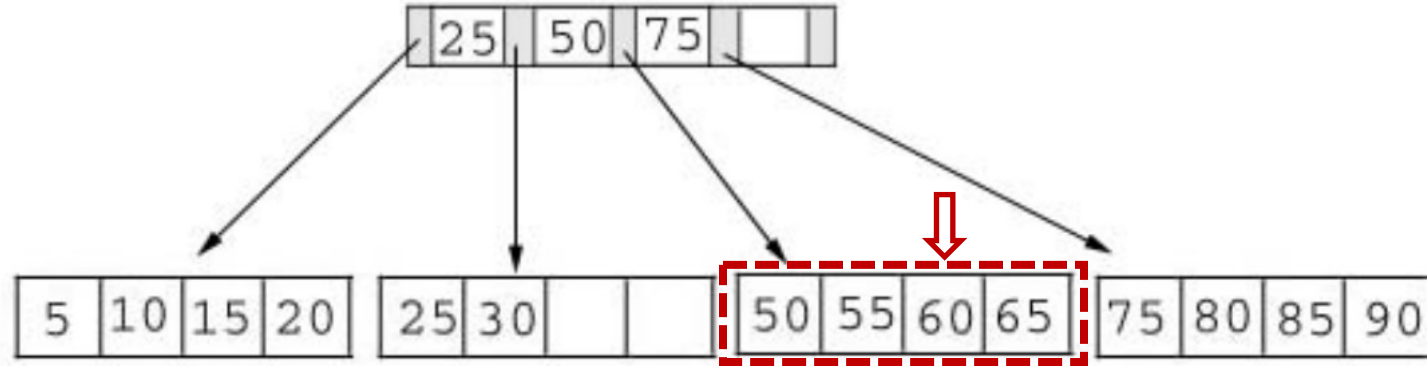
60 değerini arayalım



1. 60 değerinden küçük olan en büyük anahtardan sonraki pointera gidilir.

➤ B+ Ağacı: Arama (Search)

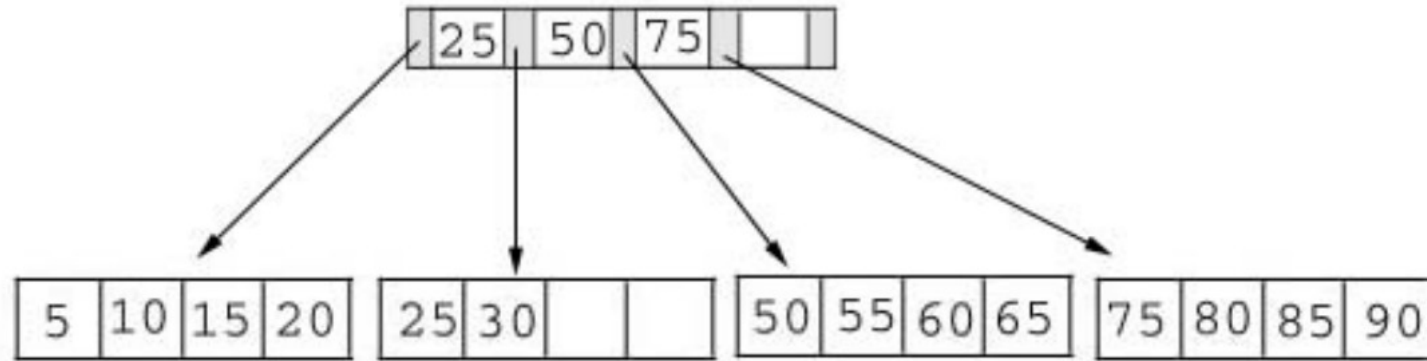
60 değerini arayalım



1. 60 değerinden küçük olan en büyük anahtardan sonraki pointera gidilir
2. Bulunan pointerin gösterdiği **düğüm** yaprak düğüm (leaf) olduğu için
 - **Aranılan değer bu düğümdedir yada yoktur.**
 - Bulunan yaprak düğüm içerisinde **kayıtlar sıra ile (sequential) taranır.**
 - Bu senaryo için 60 değeri bulunur.

➤ B+ Ağacı: Arama (Search)

60 değerini arayalım



Arama (search) işlemi gerçekleştirilirken
insert, delete ve update
işlemlerine izin verilmemektedir

➤ B+ Ağacı: insert (Ekleme)

1- Eklenmesi gereken kayıt için (düğüm) uygun düğüm aranır

2-Eğer bulunan yaprak düğüm maximum depolayabileceği değerden az kayıt içeriyorsa kayıt sırası dikkate alınarak ekleme yapılır (işlem tamamlanır)

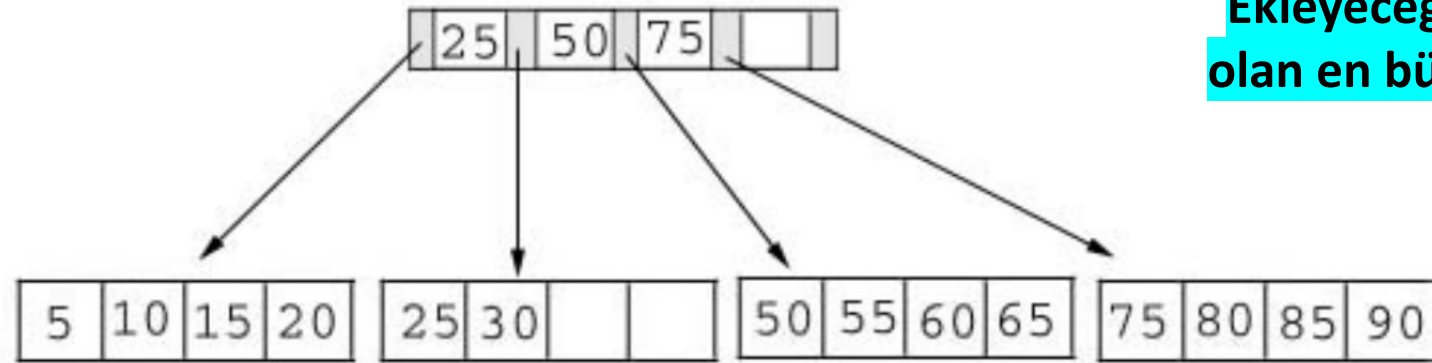
3- Eklenecek değer için düğümde yer yoksa

- Düğüm ikiye bölünür, yeni bir yaprak (leaf) oluşturulur
- Elemanların yarısı sırası değişmeden oluşturulan yeni düğüme(yaprağa) eklenir.
- *Yeni oluşturulan düğümün anahtarı bir üst seviyedeki düğüme eklenir*
- *Üst seviyedeki düğüm dolu ise ikiye ayrılır ve ortadaki anahtar üst seviyeye alınır.*
- Bölünme gerektirmeyen düğüm oluşuncaya kadar devam eder.

4- Eğer root (kök) düğümünün bölünmesi gerekiyorsa tek bir degeri ve iki pointeri bulunan yeni bir root oluşturulur.

➤ B+ Ağacı: Insert (Ekleme)

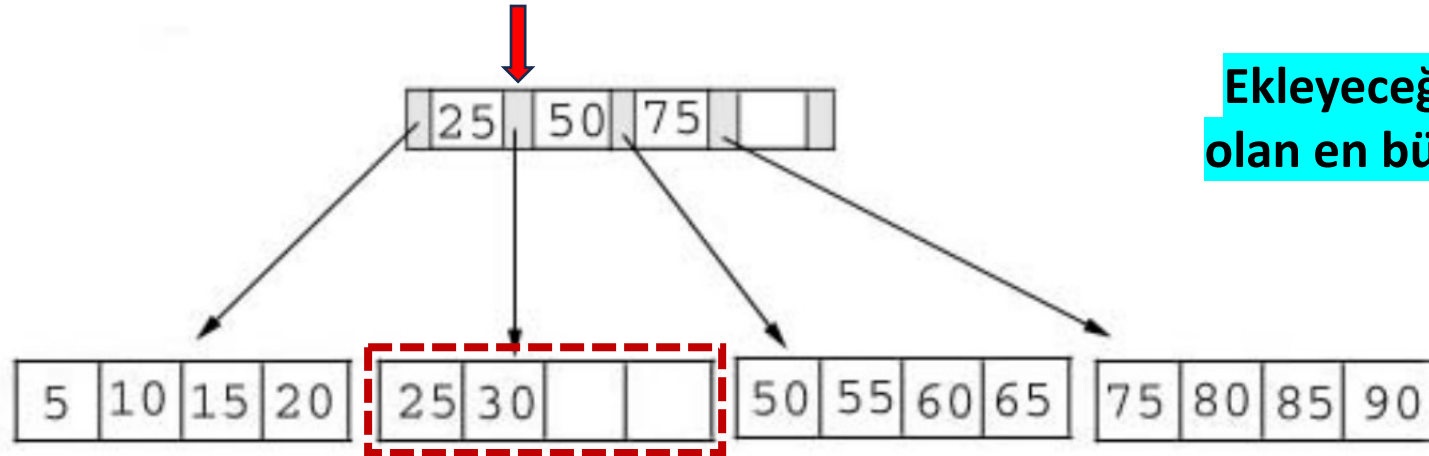
29 değerini ekleyelim



Ekleyeceğiniz değerden küçük olan en büyük değerden sonraki pointeri bul

➤ B+ Ağacı: Insert (Ekleme)

29 değerini ekleyelim



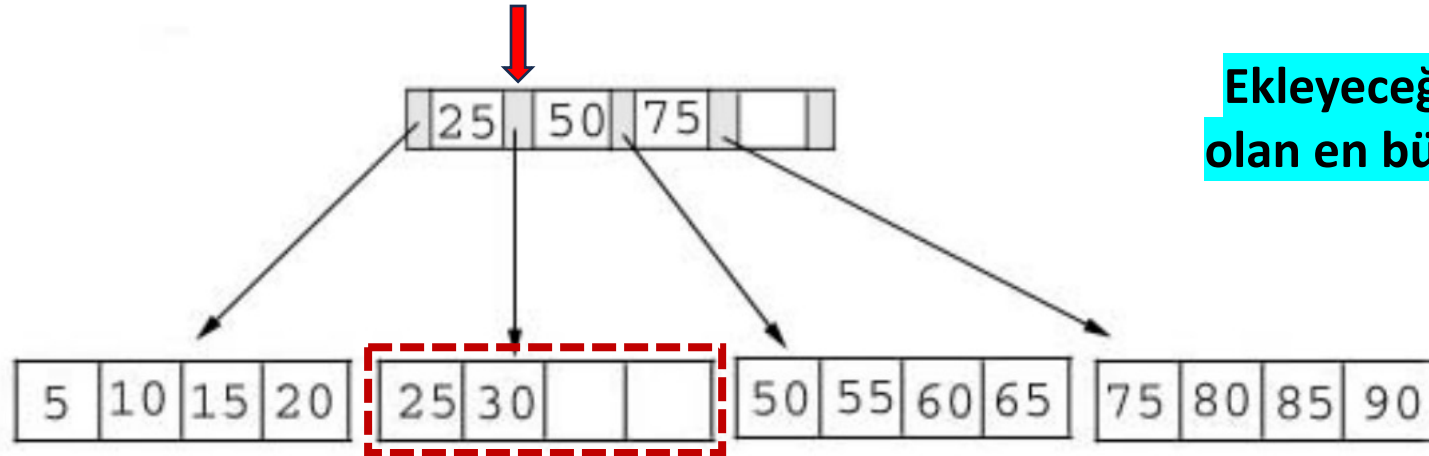
Ekleyeceğiniz değerden küçük olan en büyük değerden sonraki pointeri bul

Pointer'in gösterdiği yaprak düğüm mü?

Veri eklenecek yer var mı?

➤ B+ Ağacı: Insert (Ekleme)

29 değerini ekleyelim



Ekleyeceğiniz değerden küçük olan en büyük değerden sonraki pointeri bul

Pointer'in gösterdiği yaprak düğüm mü?

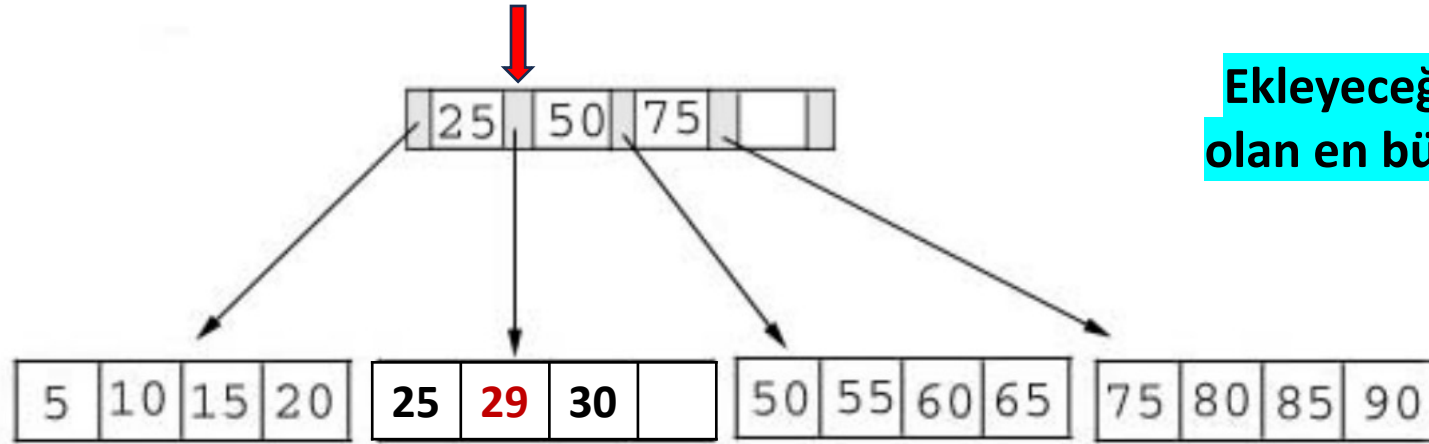


Veri eklenecek yer var mı?



➤ B+ Ağacı: Insert (Ekleme)

29 değerini ekleyelim

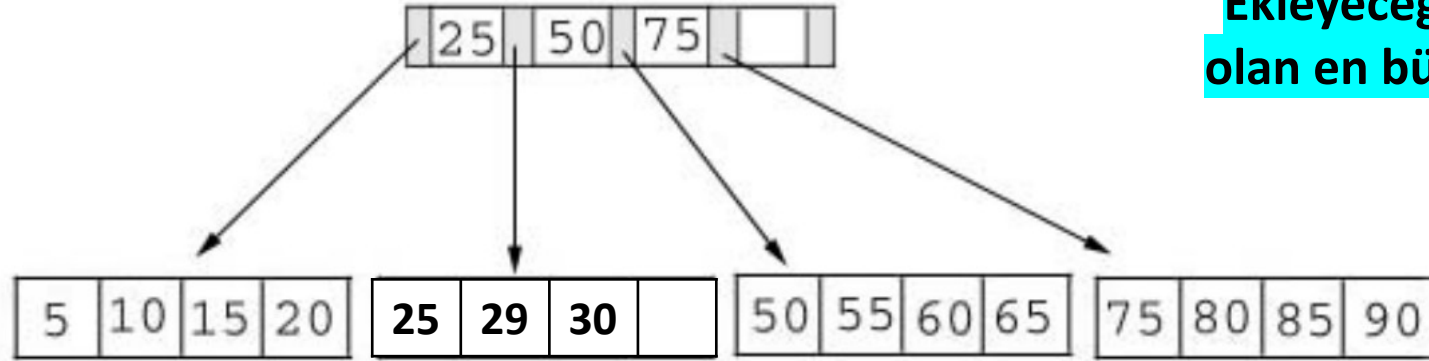


Ekleyeceğiniz değerden küçük olan en büyük değerden sonraki pointeri bul

Yaprak düğümde yer olduğundan 29 eklendi ve
Yaprak düğümde bulunan sayılar kendi içinde sıralandı

➤ B+ Ağacı: Insert (Ekleme)

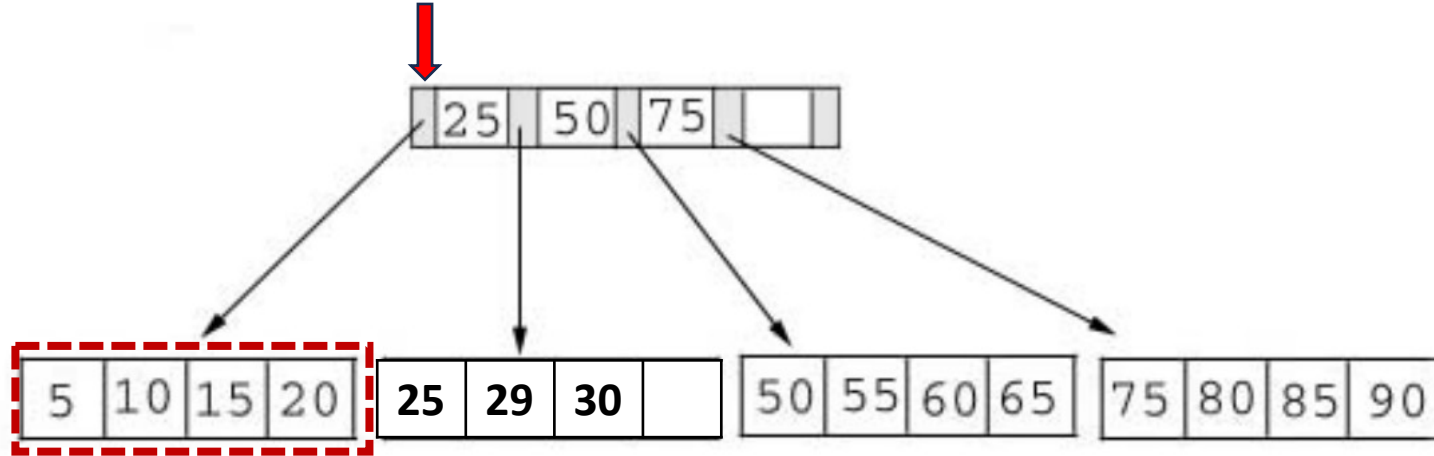
23 değerini ekleyelim



Ekleyeceğiniz değerden küçük olan en büyük değerden sonraki pointeri bul

➤ B+ Ağacı: Insert (Ekleme)

23 değerini ekleyelim



Pointer'ın gösterdiği
yaprak düğüm mü?

?

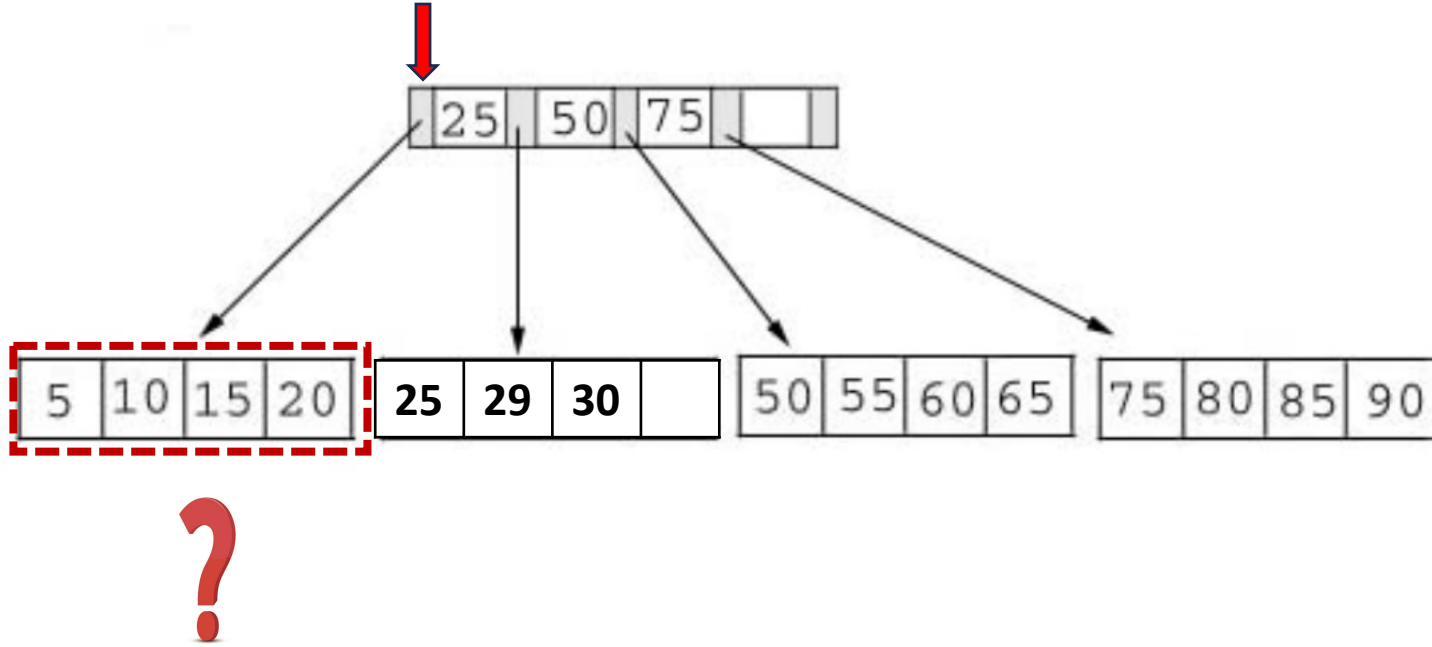
Veri eklenecek
yer var mı?

?

➤ B+ Ağacı: Insert (Ekleme)

23 değerini ekleyelim

Yaprak düğümde
değerin eklenmesi için
eklenmesi gereken
alan yok !



Pointer'ın gösterdiği
yaprak düğüm mü?

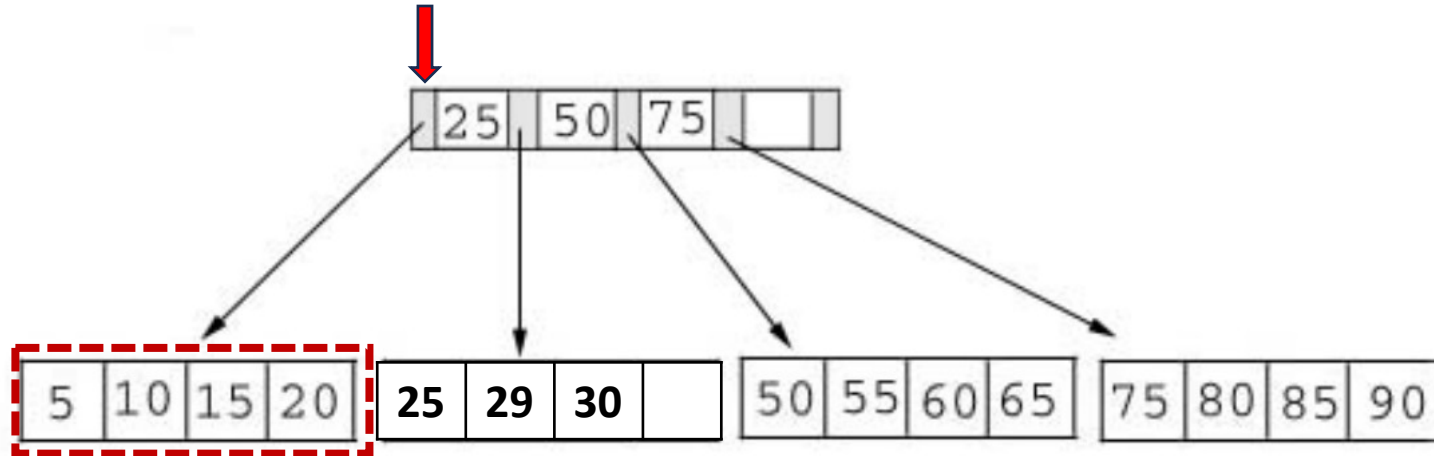


Veri eklenecek
yer var mı?

HAYIR

➤ B+ Ağacı: Insert (Ekleme)

23 değerini ekleyelim



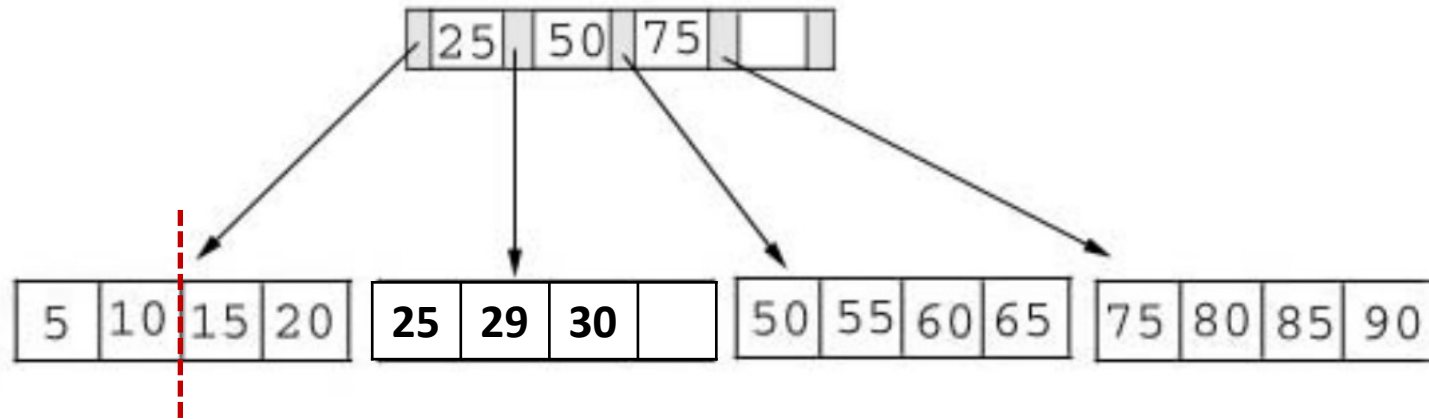
Düğüm Bölünecek!

Düğüm bölünme
kuralını açıklayınız



➤ B+ Ağacı: Insert (Ekleme)

23 değerini ekleyelim



5 | 10 | |

?

15 | 20 | |

?

Düğüm Bölündü

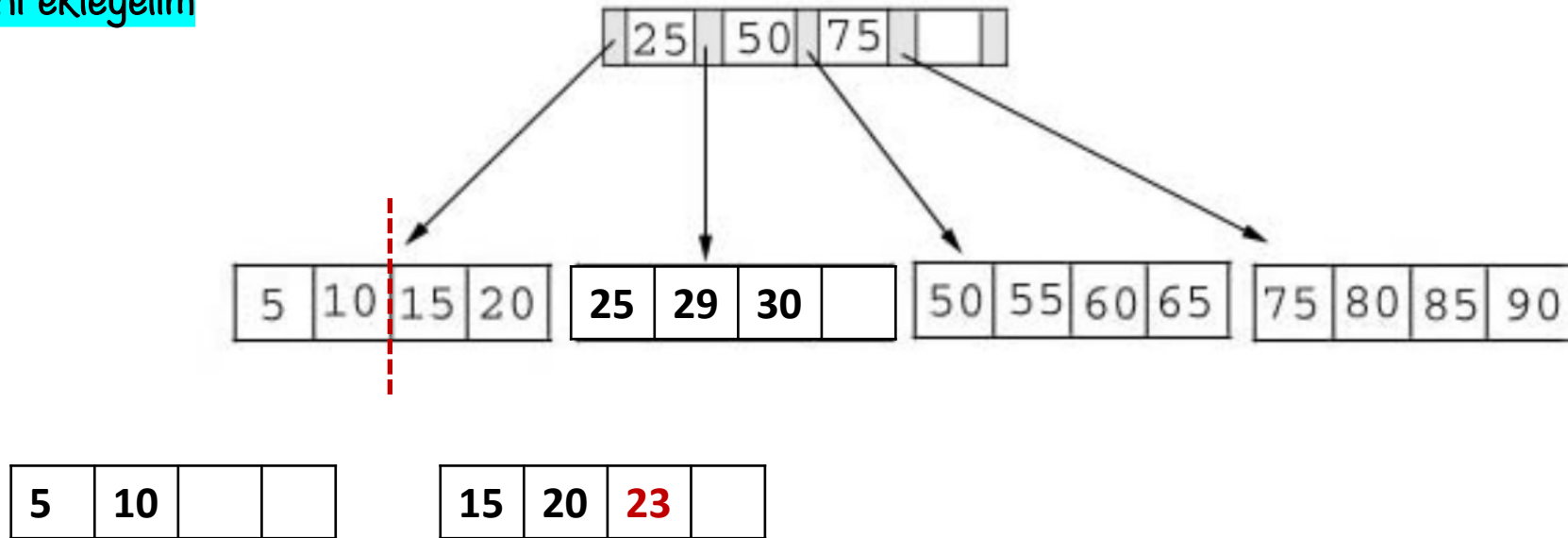
23

Hangi Düğüme Eklenmeli ?

Sayfa(düğüm) oluşturma kuralı
 $d \leq \text{düğümdeki elaman sayısı} \leq 2d$

➤ B+ Ağacı: Insert (Ekleme)

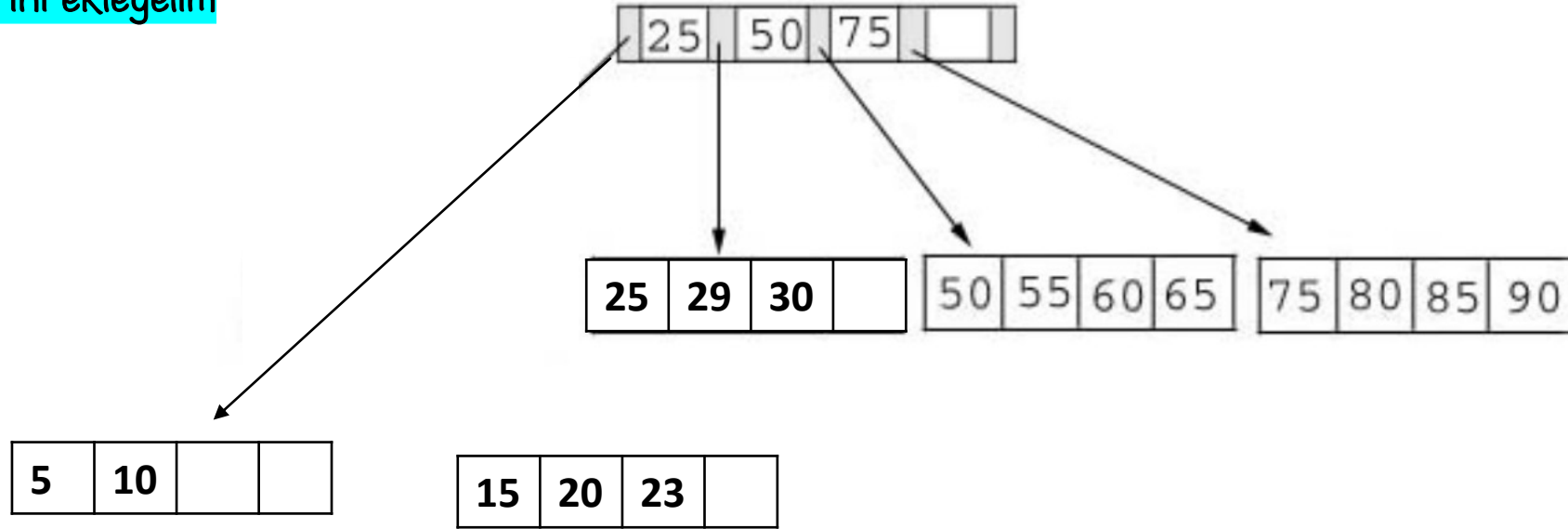
23 değerini ekleyelim



Eklenecek değere göre
sayfaların anahtar değeri dikkate alınarak
uygun olan sayfaya yeni değer eklenir.

➤ B+ Ağacı: Insert (Ekleme)

23 değerini ekleyelim

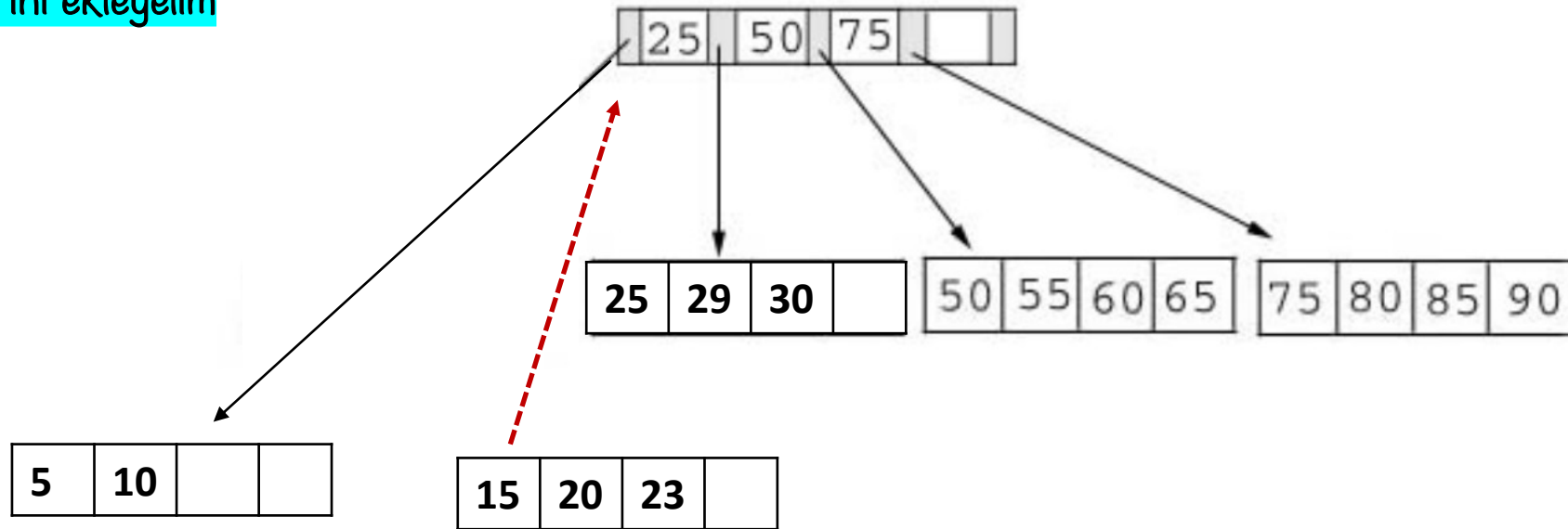


Oluşturulan yeni düğümün
pointeri yok !

?

➤ B+ Ağacı: Insert (Ekleme)

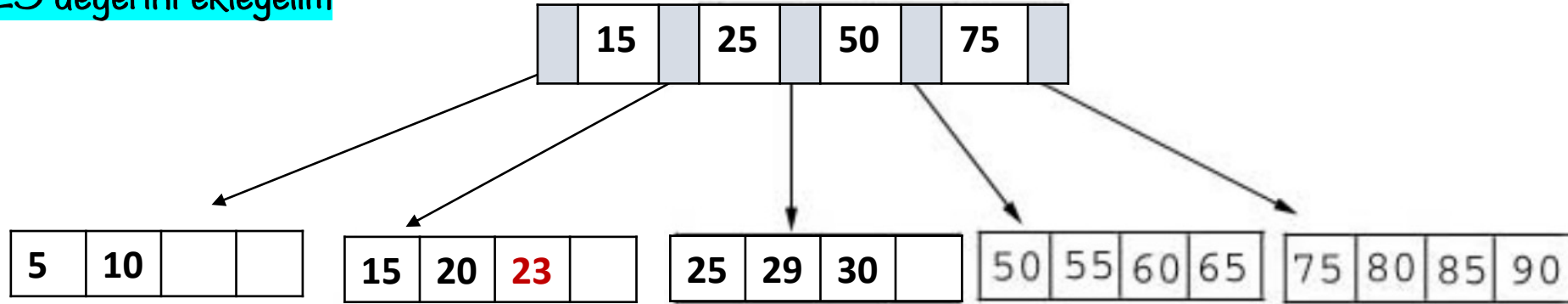
23 değerini ekleyelim



Oluşturulan yeni düğüm için
bir üst seviyeye yeni oluşturulan sayfa için
anahtar ve pointer eklenecek

➤ B+ Ağacı: Insert (Ekleme)

23 değerini ekleyelim



23 değerini ekledikten sonra elde edilen yeni ağaç

Dinlediğiniz için Teşekkürler...

İyi çalışmalar ve başarılar dilerim ...
